What is fork()?

fork() is a POSIX system call that **creates a new process** by duplicating the calling (parent) process. The new process is the **child**.
After fork() both processes continue execution from the point of the call, but each has its own memory space (copy-on-write), file descriptors, and execution context.

- **Two processes** after the call: parent and child.

Return value

- **In the parent, fork() returns the child PID (>0).**
- **In the child, fork() returns 0.**
- **On error, fork() returns -1 in the parent and no child is created.**

After a successful fork(), **two processes run concurrently**:

  - Both execute the same code following the fork()
  - But they receive **different return values**:
    - 0 in the **child process**
    - The **child's PID** in the **parent process**
- The **init** (or **systemd**) process is the **ancestor** of all processes in Linux.

The **pstree** command displays processes in a **hierarchical tree format**, showing parent–child relationships.

Step 1: Write the C Program
Create a file named fork_example.c:

```
#include <stdio.h>
#include <unistd.h> /*POSIX standard header file in C that provides access to operating system services*/
#include <sys/types.h>  /*POSIX / UNIX header file that defines data types used in system calls and low-level OS programming.*/

int main() {
   pid_t pid; //declares a variable pid of type pid_t, which is used to store a process ID.
   pid = fork();   // create a new process

   if (pid < 0) {
      perror("Fork failed");
      return 1;
   }
   else if (pid == 0) {
      // Child process
      printf("Child Process:\n");
      printf("   Child PID: %d\n", getpid());/* get the Process ID (PID) of the currently running process.*/
      printf("   Parent PID: %d\n", getppid()); /*Parent Process ID (PPID) of the current process.*/
   }
   else {
      // Parent process
      printf("Parent Process:\n");
      printf("   Parent PID: %d\n", getpid());
      printf("   Child PID: %d\n", pid);
```

```
        sleep(300);  // delay so pstree can show both processes
    }
    return 0;
}
```

Step 2: Compile and Run

gcc fork_example.c -o fork_example

./fork_example

Sample Output:

Parent Process:
  Parent PID: 3412
  Child PID: 3413
Child Process:
  Child PID: 3413
  Parent PID: 3412

Step 3: Observe the Process Tree

While the program is running (especially during the 5-second sleep), run in another terminal:

pstree -p 3412 */*Displaying running processes in a tree (hierarchical) structure along with their Process IDs (PIDs)*/*

or, to view the entire tree from the init process: */****init process*** *is the* ***first user-space process*** *started by the Linux kernel during system boot.The init process is the first process created by the Linux kernel (PID 1) and is the ancestor of all other processes.*/*

pstree -p 1

Displaying the process tree starting from the init process (PID 1), along with PIDs.
pstree → shows processes in tree (hierarchical) format
-p → shows Process IDs (PIDs)
1 → PID of **init / systemd** process

Sample Output (Simplified):

```
systemd(1)─┬─bash(3200)─┬─fork_example(3412)─┬─fork_example(3413)
           │            └─pstree(3420)
```

Explanation:

- systemd(1) is the **init process** (PID 1)
- bash(3200) is your shell (parent of your program)
- fork_example(3412) is the **parent process** created by you
- fork_example(3413) is the **child process** created by fork()

Step 4: Understanding Process Hierarchy

- Every process (except **init/systemd**) is a child of another process.
- The **fork()** system call duplicates the parent process.
- Using **pstree**, you can visualize this relationship easily.

Step 5: Verify with ps command

You can also verify the parent-child relationship using ps:
ps -ef | grep fork_example
Sample Output:

```
user    3412 3200  0 11:20 pts/0    00:00:00 ./fork_example
user    3413 3412  0 11:20 pts/0    00:00:00 ./fork_example
```

This shows:

- The **child (3413)** has **parent 3412**
- The **parent (3412)** has **parent 3200** (bash)

Step 6: After Process Termination

Once both processes finish, re-running pstree will no longer show them — they were temporary.